المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵉⵙⵉⵏⵉ - ⴰⵎⵜ ⵎⵓⵍⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Ecole nationale des sciences appliquées de Beni-Mellal

Niveau : 1ère année
Filière : IACS

TP1: Application Monolithique

Réalisé par:
TOHANI Marwa

Encadré par:
Pr. ElBaghzaoui

1. Introduction

Ce TP consistait à mettre en place un monolithe simple pour comprendre les bases de Spring Boot : l'initialisation du projet, la persistance des données avec PostgreSQL (via Docker) et l'exposition d'endpoints via un Controller.

2. Architecture du projet :

Nous avons suivi l'architecture standard en 4 couches recommandée :

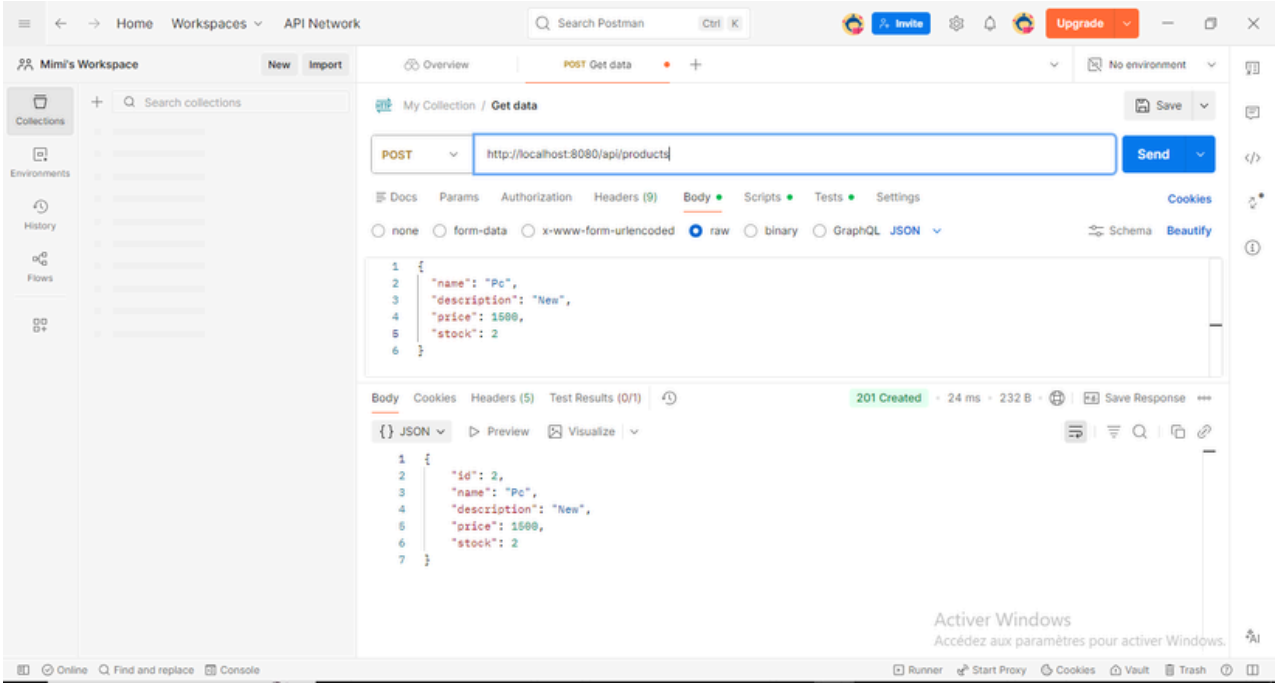
- Entity (Product) : Définition du modèle avec des contraintes de validation (@NotBlank, @Positive).
- Repository (ProductRepository) : Utilisation de Spring Data JPA pour l'accès aux données.
- Service (ProductService) : Centralisation de la logique métier et gestion des transactions (@Transactional).
- Controller (ProductController) : Exposition des routes API (/api/products).

3. Tests et Validation :

On a fait un test avec postman pour tester les routes

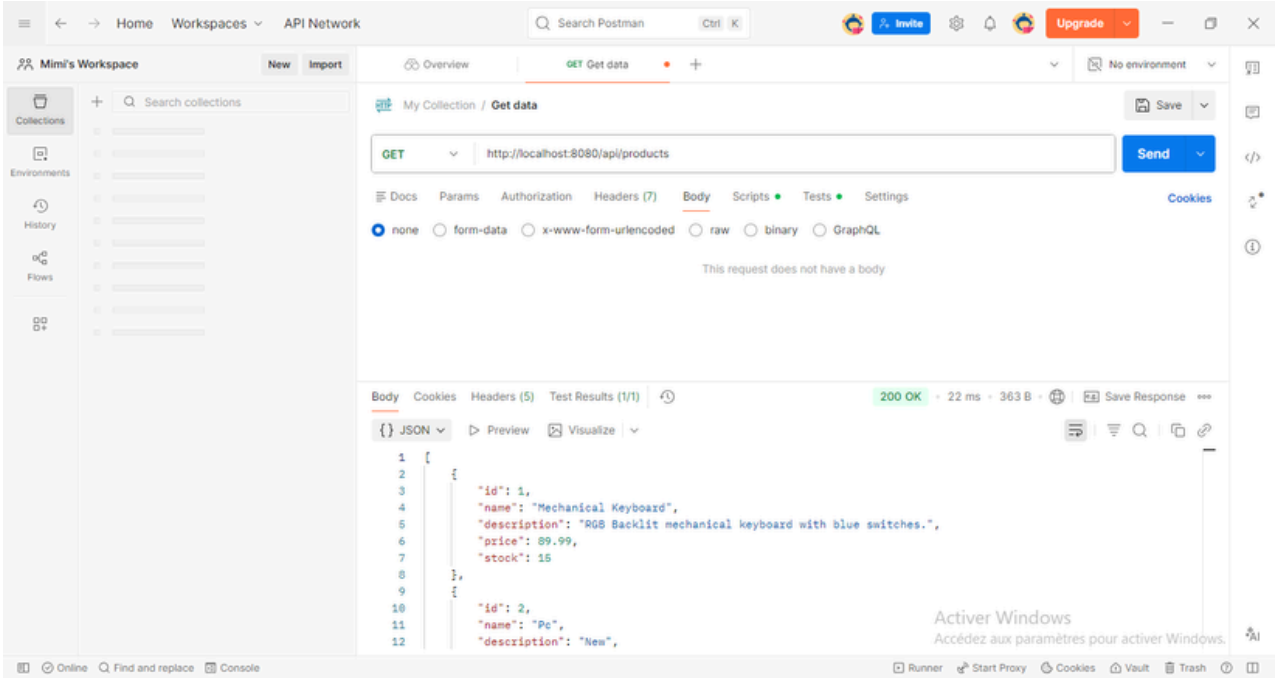
A. Création d'un Produit (POST)

URL : http://localhost:8080/api/products
Action : Envoi d'un JSON contenant name, description, price et stock.



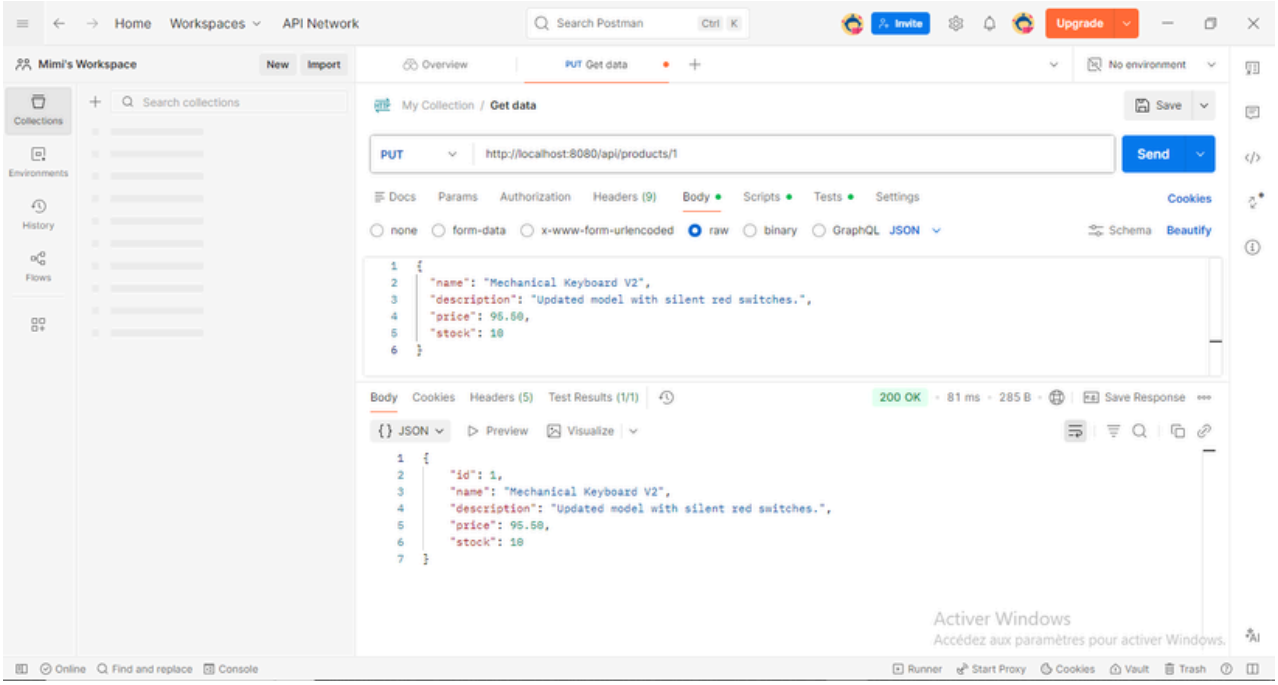
B. Liste des Produits (GET)

URL : http://localhost:8080/api/products
Action : Récupération de la liste complète des produits enregistrés en base.



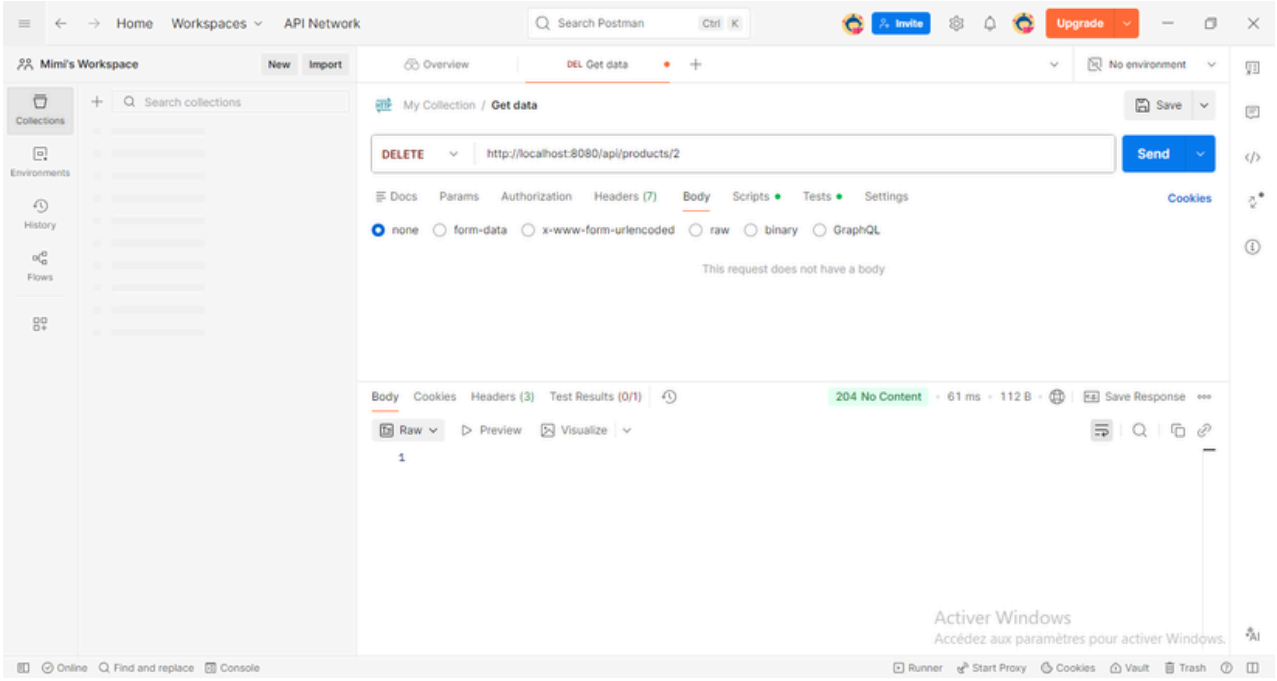
C. Mise à jour d'un Produit (PUT)

URL : http://localhost:8080/api/products/{id}
Action : Modification du prix ou du stock d'un produit existant.



C. Supprimer un produit (DELETE)

URL : http://localhost:8080/api/products/{id}
Action : Suppression d'un produit



3. Exercice (category):

Conformément à l'exercice final du sujet, j'ai ajouté :

- Une entité Category (id, name).
- Une relation ManyToOne dans la classe Product (plusieurs produits peuvent appartenir à une même catégorie).

- **Créer l'entité Category**

```
package com.ecommerce.monolith.product.model;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.util.List;

@Entity
@Getter @Setter @NoArgsConstructor
@AllArgsConstructor @Builder
public class Category {
   @Id
   @GeneratedValue(strategy =
GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "Category name is required")
   private String name;

   @OneToMany(mappedBy = "category")
   private List<Product> products;
}
```

- ## Modifier l'entité Product

Ajoute la relation ManyToOne dans ton fichier Product.java existant :

```
@ManyToOne
@JoinColumn(name = "category_id") // This creates the foreign key in the database
private Category category;
```

- ## Créer le CategoryRepository

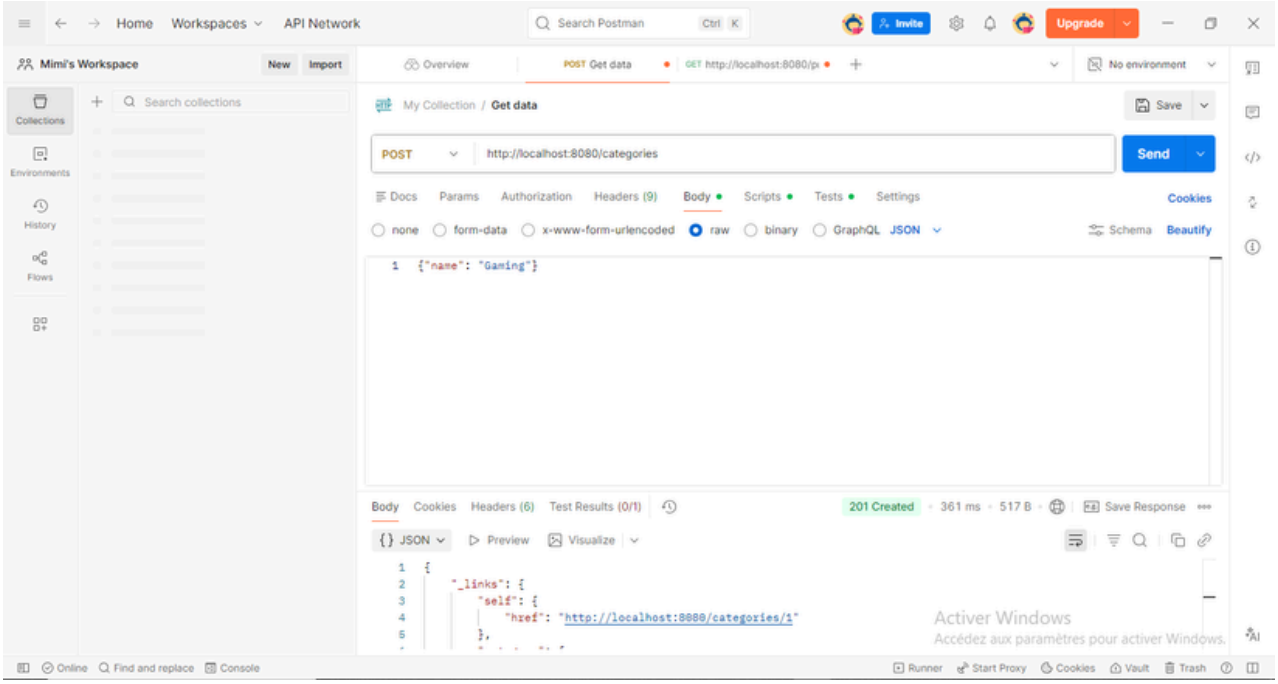
```
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.rest.core.annotation.RepositoryRestResource;

@RepositoryRestResource(collectionResourceRel = "categories", path = "categories")
public interface CategoryRepository extends JpaRepository<Category, Long> {
}
```

- ## Créer le CategoryRepository

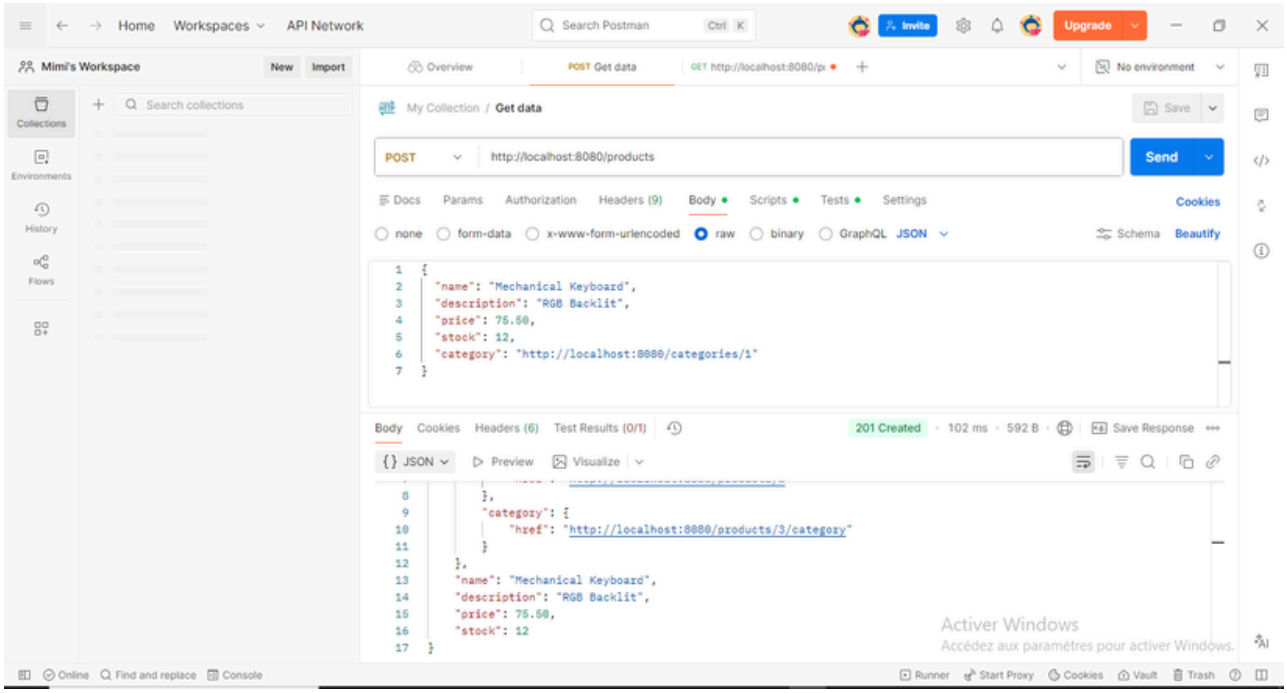
```
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Category;
import
```

- ## Test Postman

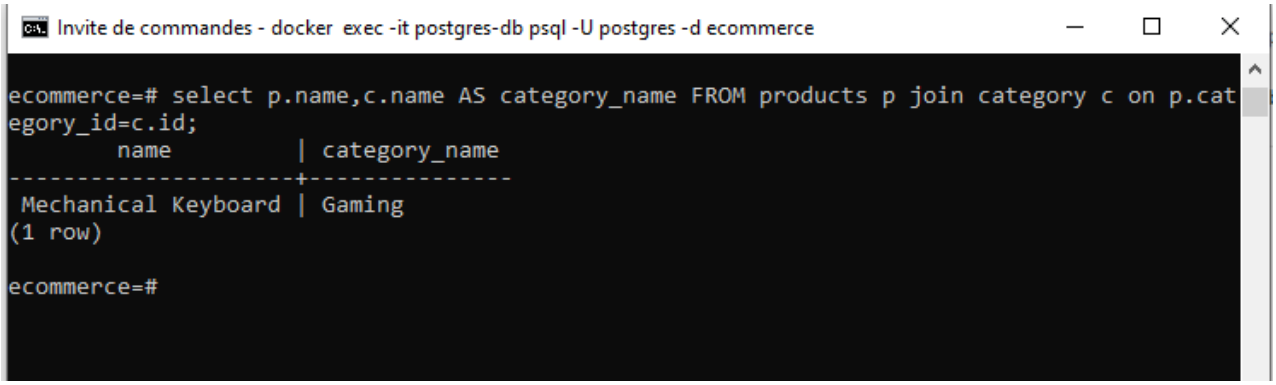
URL : http://localhost:8080/api/categories
Action : Post {"name": "Gaming"}



URL : http://localhost:8080/api/products
Action : Post



- **Test Base de données**



4. Conclusion :

Ce TP a permis de maîtriser le cycle de vie d'une requête dans Spring Boot. L'utilisation de Lombok et des annotations JPA réduit considérablement le code répétitif (boilerplate), permettant de se concentrer sur la logique métier.